

Informe

Programacion I

The logo of the Universidad Tecnológica Nacional (UTN) is displayed. It consists of a stylized sun-like symbol on the left, followed by a vertical line, and then the letters 'UTN' in a bold, sans-serif font.

Estudiantes:

- Pablo Funes
- Valentino Rossi

Profesores:

- Cinthia Rigoni
- Ariel Enferrel

Fecha de entrega: XX/06/2026

Indice

Introduccion	-----	3
Base de datos:	-----	3
Variable	-----	3
Listas	-----	3
Diccionarios	-----	4
Estructuras condicionales:	-----	4
if	-----	4
elif	-----	4
else	-----	4
Estructuras repetitivas:	-----	5
while	-----	5
for	-----	5
Funciones	-----	6
Manejo de errores:	-----	6
try	-----	6
except	-----	6
finally	-----	7
Tipos de errores	-----	7
Manejo de archivos	-----	8
Otras funciones nativas de python:	-----	8
strip()	-----	8
split()	-----	9
.join()	-----	9
Librearias:	-----	9
csv	-----	9
os	-----	9
questionary	-----	10
Bibliografia	-----	11

Introducción:

Todo lo explicado en este informe, fue usado en la practica, esto incluye funciones internas de python y externas. Se detalla cada uso con el fin de dar a entender al lector lo que puede reconstruir teniendo en cuenta la explicación del informe. Este informe busca que sea más una guía que una simple explicación.

Variables:

Son datos guardados en base a un nombre. Estos datos pueden ser strings, int, floats y bool.

```
variable = "Hola"
```

Variable con un dato string.

```
variable = 1.5
```

Variable con un dato float.

```
variable = 10
```

Variable con un dato int.

```
variable = True
```

Variable con un dato bool.

A las variables que posean int y floats como dato. Se les puede hacer operaciones matematicas.

Operador	Función
+	Suma
-	Resta
*	Multiplicación
/	División
//	División entera
%	Módulo (resto)
**	Potencia

Ejemplo N°1:

```
a = 10
b = 5

resta = a - b
print(resta) # salida: 5
```

Ejemplo N°2:

```
a = 5
b = 10

multiplicacion = a * b
print(multiplicacion) # salida 50
```

Estas mismas variables se pueden decir dentro de un print, usando print(f''):

```
division = a / b
print(f"El resultado es: {division}")
```

Listas:

Los elementos de una lista son objetos de Python arbitrarios. Las listas se forman colocando una lista de expresiones separadas por comas entre corchetes. Es decir:

```
lista = ["Hola", "Adios", 80, 5.5]
```

```
variable1 = "Saludos"
variable2 = 80

lista = [variable1, variable2]
```

Diccionarios:

Estos representan conjuntos finitos de objetos indexados por valores casi arbitrarios. Los únicos tipos de valores no aceptables como claves son valores que contienen listas o diccionarios u otros tipos mutables que se comparan por valor en lugar de por identidad de objeto, la razón es que la implementación eficiente de los diccionarios requiere que el valor hash de una clave permanezca constante. Los tipos numéricos utilizados para las claves obedecen las reglas normales para la comparación numérica: si dos números se comparan igual (por ejemplo, 1 y 1.0) entonces se pueden usar indistintamente para indexar la misma entrada del diccionario. Es decir:

```
diccionario = {"hola": 4}
diccionario2 = {"adios": "saludo"}
```

Llave Valor

Para recorrer un diccionario podemos usar `.keys()` para las llaves, `.values()` para los valores y `.items()` para los dos:

```
diccionario = {"hola": 4, "rayos": 10, "santos": 80}

claves = diccionario.keys()
valores = diccionario.values()
claves_valores = diccionario.items()

print(list(claves))
print(list(valores))
print(list(claves_valores))

#salida:
#[ 'hola', 'rayos', 'santos' ]
#[ 4, 10, 80 ]
#[ ('hola', 4), ('rayos', 10), ('santos', 80) ]
```

if, elif y else:

Python usa la palabra clave `if` para definir una condición, y opcionalmente se pueden agregar `else` y `elif` para manejar distintos casos. Por medio de ellas es posible que el programa defina qué bloques de código ejecutar primero o directamente no hacerlo.

Python usa la indentación para indicar que pertenece a un bloque `if`:

```
variable = 0
if variable == 0:
    print("La variable es igual a cero.")
elif variable == 1:
    print("La variable es igual a uno.")
else:
    print("La variable no es igual a cero ni a uno.")
```

Las indentaciones serían esos espacios que tiene el `print`. Es como si fuera una sangría.

Poseemos operadores que nos sirven para poner más condiciones de diferentes formas:

Operador	Definición	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	True
<code>!=</code>	Distinto de	<code>5 != 3</code>	True
<code>></code>	Mayor a	<code>5 > 8</code>	False
<code><</code>	Menor a	<code>3 < 1</code>	False
<code>>=</code>	Mayor o igual a	<code>5 >= 4</code>	True
<code><=</code>	Menor o igual a	<code>4 <= 2</code>	False
<code>and</code>	Y, solo da True si todas son True.	<code>True and False</code>	False
<code>or</code>	O, solo da False si todas son False.	<code>False or True</code>	True
<code>not</code>	No, Invierte el valor.	<code>Not False</code>	True

While “variable bool”:

Es un ciclo el cual se repite siempre y cuando la variable devuelva “True”. En el caso de que la variable devuelva “False” el ciclo no se ejecuta.

```
while True:  
    print("Hola")
```

Se da un ciclo infinito, debido a que no se cambia el valor del True.

```
while False:  
    print("Hola")
```

No se ejecuta el código indentado al while debido a que el valor es False.

```
contador = 3  
while contador != 0:  
    print("Hola")  
    contador -= 1
```

El código se ejecuta hasta que la variable “contador” no sea diferente a cero. No se da un bucle infinito debido a que con cada vuelta se le resta uno a “contador”.

```
while True:  
    print("Adios")  
    break
```

En un ciclo while True podemos usar “break” para detenerlo

Ciclo for:

El bucle for permite repetir una acción un número determinado de veces. A diferencia de while, acá sabemos de antemano cuántas veces se repetirá el código.

```
for variable in range (rango_inicial, rango_final):  
    # salida 1  
    # salida 2  
    # salida 3
```

```
for i in range (0, 4):  
    print(i)  
# salida:  
# 0  
# 1  
# 2  
# 3
```

Funciones:

Una función es un bloque de código reutilizable que realiza una tarea específica. En lugar de escribir el mismo código varias veces, podemos definir una función y llamarla cuando sea necesario.

Esto sirve bastante para tener un código más ordenado.

```
def saludo():  
    print("Hola.")
```

De esta forma, nosotros definimos una función. En este caso la función "saludo"

```
saludo()
```

Para llamar a una función, escribimos su nombre y adelante le ponemos parentesis.

Las funciones pueden trabajar con datos ya existentes. Solo hace falta llamar ese parámetro para la función:

```
def suma(a, b):  
    resultado = a + b  
    return resultado
```

```
a = 5  
b = 10  
  
valor = suma(a, b)  
print(valor)  
  
# salida:  
# 15
```

En este caso, a la función suma le dijimos que use los parámetros "a" y "b" para que realice una simple suma. A la hora de usar la función, debemos de llamar esos parámetros. Colocando el nombre de las variables. En el caso de que no las llamemos, nos dará un error por falta de error.

Los parámetros recibidos no tienen que llevar si o si el mismo nombre que las variables. También podemos darle un valor sin necesidad de una variable.

```
def saludar(nombre):  
    print(f"Hola {nombre}")
```

```
saludar("Juanita")  
#salida:  
# Hola Juanita
```

try except:

El bloque try contiene el código que puede lanzar una excepción. Una excepción es un tipo especial de error que ocurre mientras el programa se está ejecutando.

```
try:  
    # Código que podría fallar:  
    suma = 5 + 2  
except error_posible:  
    # Parte del código que se ejecuta al saltar un error.  
    print("No es posible realizar esa suma.")
```

Se pueden usar múltiples except y también except no especificados:

```
try:  
    suma = "5" + 2  
except TypeError:  
    print("No es posible realizar una suma con string e int.")  
except:  
    print("Ocurrió un error no especificado")
```

Tipos de errores:

◆ZeroDivisionError

Se produce cuando intentamos dividir un número por cero.

```
try:
    division = 8 / 0
except ZeroDivisionError:
    print("No es posible dividir por cero.")
```

◆ValueError

Ocurre cuando intentamos convertir datos a un tipo incompatible. Por ejemplo, convertir letras a número.

```
try:
    int(input("Ingrese su letra inicial por favor: "))
except ValueError:
    print("Las letras no se pueden convertir a numeros.")
```

◆FileNotFoundError

Aparece cuando se intenta abrir un archivo que no existe.

```
try:
    archivo = open("archivo.txt", "r")
except FileNotFoundError:
    print("No se encontro un archivo con ese nombre.")
```

◆KeyError

Se lanza cuando tratamos de acceder a una clave que no existe en un diccionario.

```
granja = {"materia prima": "trigo", "producto": "harina"}
try:
    print(granja["precio"])
except KeyError:
    print("No se encontro una clave con ese nombre.")
```

◆IndexError

Se genera al intentar acceder a una posición que no existe en una lista o tupla.

```
lista = ["Hola", "Bola", "Sola", "Ola"]
try:
    print(lista[4])
except IndexError:
    print("No hay ningun elemento en el index 4")
```

◆TypeError

Se lanza cuando se realizan operaciones entre tipos de datos incompatibles.

```
try:
    multiplicacion = "10" * 5
except TypeError:
    print("No se puede realizar calculos matematicos entre strings e ints.")
```

◆PermissionError

Se genera cuando no hay permisos suficientes para abrir el archivo.

```
try:
    archivo = open("archivo.txt", "r")
except PermissionError:
    print("No se puede acceder al archivo debido a falta de permisos.")
```

Además de los bloques try y except, existe un bloque especial llamado finally. Este bloque se ejecuta siempre, sin importar si ocurrió una excepción o no.

```
try:
    suma = 10 + 67
except TypeError:
    print("No se pueden hacer operaciones matematicas con valores de diferente tipo.")
finally:
    print("Intento de tarea realizado.")
```


Manejo de archivos:

Modos de apertura:

"r" Abre un archivo para lectura en modo texto. El archivo debe existir, si no, lanza un error.	"rb" Abre un archivo para lectura en modo binario.	"r+" Abre un archivo para lectura y escritura en modo texto. El archivo debe existir. No borra su contenido.
"w" Abre un archivo para escritura en modo texto. Si el archivo ya existe, borra su contenido. Si no existe, lo crea.	"wb" Abre un archivo para escritura en modo binario. Si el archivo ya existe, se borra. Si no existe, se crea.	"w+" Abre un archivo para lectura y escritura, en modo texto. Sobrescribe el contenido del archivo si ya existe.
	"a" Abre un archivo para agregar contenido al final. Si el archivo no existe, lo crea. No borra el contenido previo.	

Para su uso, basta con nombrar al archivo como una variable, escribir `open(nombre_archivo, modo)`:

```
archivo = open("archivo.txt", "r")
```

Los archivos que podemos abrir son varios, como por ejemplo: .txt .csv .json y otros más.

En el modo van los ya mencionado "modos de apertura": "r", "w", "a", entre otros.

Para cerrar un archivo, usamos `.close()`

```
archivo.close()
```

Al cerrar un archivo podemos evitar perdida de datos, bloqueos de sistema, etc.

Para trabajar de una forma más segura, usamos `with`. Abre el archivo y garantiza que se cerrará automáticamente al finalizar el bloque, incluso si ocurre un error dentro del mismo.

```
with open("archivo.txt", "r") as archivo:
    for line in archivo:
        print(f"{line}\n")
```

Metodos para "w"	Metodos para "r"
write() : permite escribir una sola linea.	read() : Lee todo el contenido del archivo como una unica cadena de texto
No existe un <code>writeline()</code>	readline() : Lee una sola linea del archivo.
writelines() : permite escribir varias lineas	readlines() : Lee todo el contenido del archivo como una lista de cadenas.

Otras funciones nativas de python:

◆ `strip()`:

La función `strip()` se usa para eliminar los caracteres en blanco al inicio y al final de una cadena de texto. Tambien se le pueden pasar caracteres especificos para eliminar.

```
texto = "  En esta oracion estoy poniendo espacios al inicio y al final  "
texto_sin_espacios = texto.strip()
print(texto_sin_espacios)

# salida:
# En esta oracion estoy poniendo espacios al inicio y al final
```


◆ split():

La función split() divide una cadena en una lista de partes, usando como criterio de separación un carácter específico.

```
texto = "Hola,Adios,10,Ocho"
partes = texto.split(",")
print(partes)

# salida:
# ['Hola', 'Adios', '10', 'Ocho']
```

◆ .join():

.join() es un método que pertenece a los strings y se usa para unir elementos de una secuencia (como listas o tuplas) en un solo string, usando el string que llama a .join() como separador:

```
palabras = ["Hola", "adios"]
resultado = " ".join(palabras)
print(resultado)

# salida:
# Hola adios
```

Librerías:

Las librerías son modificaciones que agregan código, con la finalidad de realizar acciones que sin ellas no podríamos dentro de python.

Para importar una librería, usamos import:

```
import libreria
```

Python posee librerías, estas librerías son clasificadas como internas, también hay librerías externas: Las que no pertenecen a python y requieren de una instalación.

Usando pip instalamos "librería" dentro de la consola. Podemos instalar librerías:

```
PS C:\Users\Usuario> pip install colorama
```

Las librerías puestas en práctica son:

◆ Librerías internas:

Python posee librerías, en el proyecto hemos aprovechado algunas de ellas:

◆ csv:

La estamos usando para interactuar con el archivo csv.

Como si fuera un archivo, lo abrimos de la misma forma que en manejo de archivos, con algunas excepciones en el código:

```
import csv

with open("archivo.csv", "r", encoding="utf-8") as archivo:
    lector = csv.reader(archivo)
```

Encoding="utf-8": nos sirve para detectar caracteres que python no detecta normalmente. Ej: Ñ, ñ, á, é, ö.

En estas también entran distintas herramientas usables, que nos permiten trabajar de ciertas maneras más cómodas.

next():

Función integrada con csv. Sirve para "saltarnos" los encabezados y trabajar solo con los datos.

```
encabezados = next(lector)
print(f"Columnas detectadas: {encabezados}")
```

◆ os:

os tiene muchas funciones, nosotros la usamos solamente para limpiar la consola.

La indicación para limpiar la consola es la siguiente:

```
import os

os.system('cls' if os.name == 'nt' else 'clear')
```

◆ **Librerías externas:**

◆ **Questionary:**

Questionary es una librería que permite el uso de las flechas del teclado para manejarse entre las opciones de un menú. El enter permite seleccionar esa opción.

Con `questionary.select().ask()` podemos crear un menú, y como devuelve un valor, se lo podemos asignar a una variable:

```
import questionary

opcion = questionary.select("¡Hola!", ["1-Hola;", "2-Adios;", "3-Callate."]).ask()

print(opcion)
```

Lo primero que va como string es la pregunta que queremos hacer, después, entre corchetes escribimos las opciones separadas por comas. Estas opciones pueden ser strings, ints, floats, etc.

Dentro de la consola se ve así:

```
? ¡Hola! (Use arrow keys)
» 1-Hola;
  2-Adios;
  3-Callate.
```

Si nosotros apretamos una sola vez la flecha de abajo, empezamos a navegar por las opciones disponibles:

```
? ¡Hola! (Use arrow keys)
  1-Hola;
» 2-Adios;
  3-Callate.
```

Si apretamos enter, entonces seleccionaremos la opción:

```
? ¡Hola! 2-Adios;
2-Adios;
PS C:\Users\Usuario>
```

El `print(opcion)` imprimió la opción seleccionada. Arriba, al lado de la pregunta podemos ver la opción seleccionada también.

Bibliografía:

Funciones y librerías nativas:

Estructuras secuenciales (variables, operadores.):

[Estructuras secuenciales part-1.pdf](#)

[Estructuras secuenciales part-2.pdf](#)

Lista:

[3. Modelo de datos — documentación de Python - 3.14.5](#)

Diccionarios:

[Diccionarios - TUPaD.docx](#)

Estructuras condicionales:

[Condicionales](#)

[Operadores Logicos](#)

[Operadores Relacionales](#)

Estructuras repetitivas:

[Repetitivas-part-1.pdf](#)

[Repetitivas-part-2.pdf](#)

Manejo de errores:

[Tipo de Errores.docx](#)

[Try Except.docx](#)

Manejo de archivos:

[1 - Introducción Archivos .pdf](#)

[Apertura, lectura, escritura, cierre de archivos.pdf](#)

[Apunte - archivos.pdf](#)

Otras funciones nativas:

[strip\(\).split\(\).join\(\).pdf](#)

Librería questionnaire:

[questionnaire · PyPI](#)

Datos de países:

[Microsoft Copilot](#)

Con informaciones adicionales añadidas en base a lo aprendido en clases de programación I